

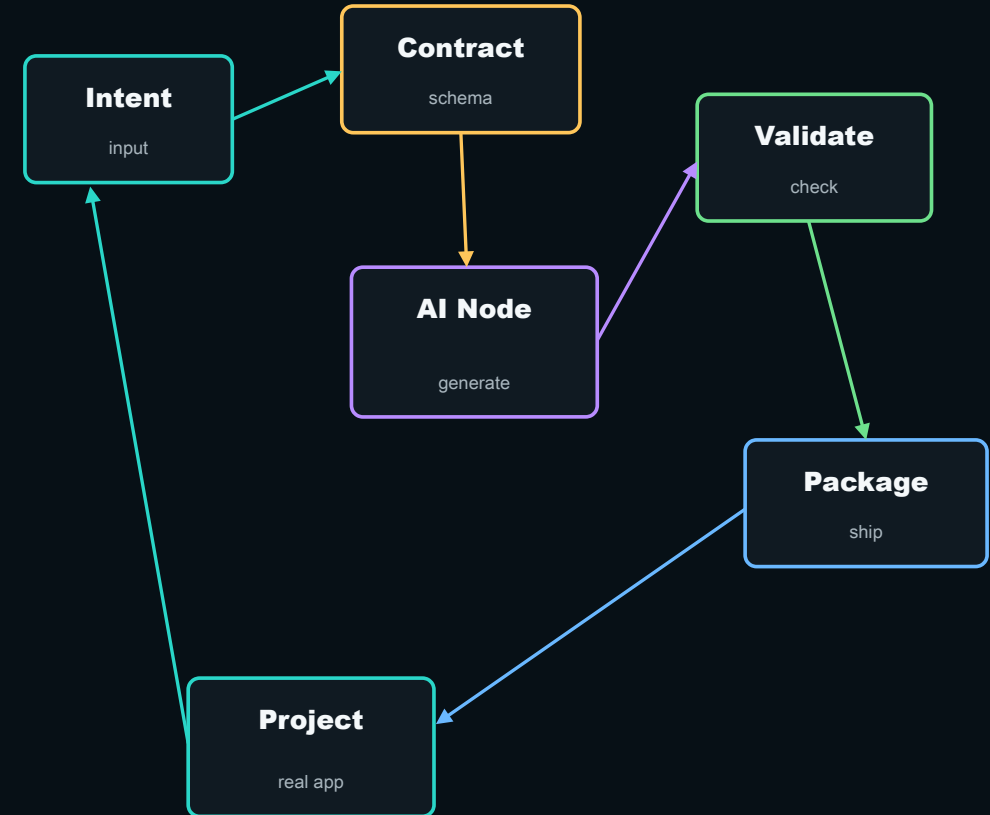
Harness AI with a DAG

From impressive AI outputs to repeatable enterprise execution.

VL

DAG

AI Assistant



The symptom: context rot

Longer memory helps, but it does not define the process.

Current fixes

Longer context
Retrieval
Memory summaries
Prompt rules
Guardrails

What is still missing

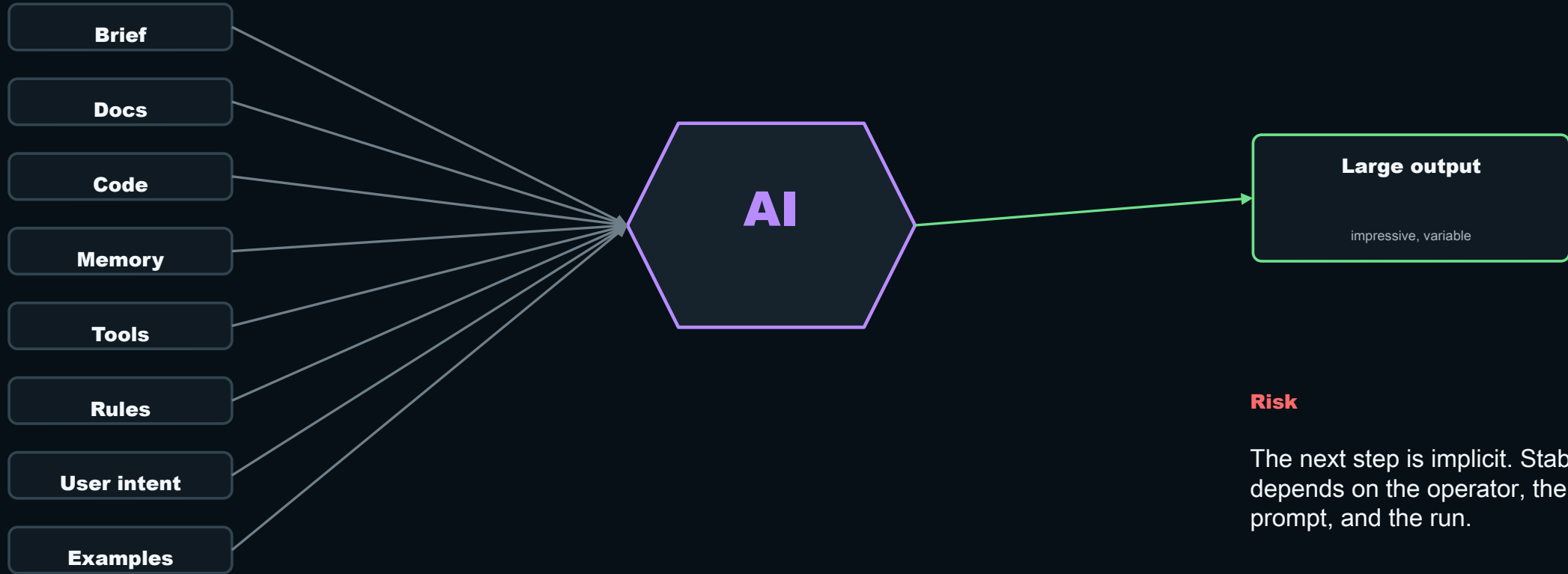
Execution order
Dataflow contracts
Quality gates
Repair loops
Human handoffs

The core answer

Plan the whole harness as a runnable graph,
then let AI operate inside that graph.

One AI node is powerful, but unmanaged

A big output is not the same as a complete process.

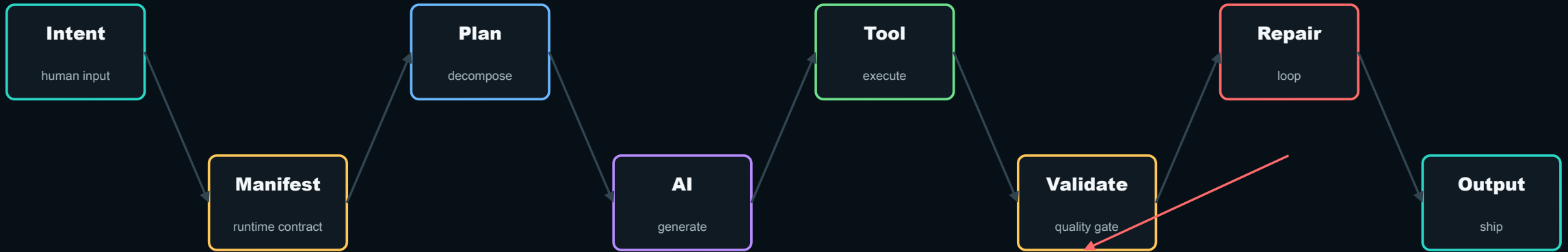


Risk

The next step is implicit. Stability depends on the operator, the prompt, and the run.

A DAG turns AI into a process

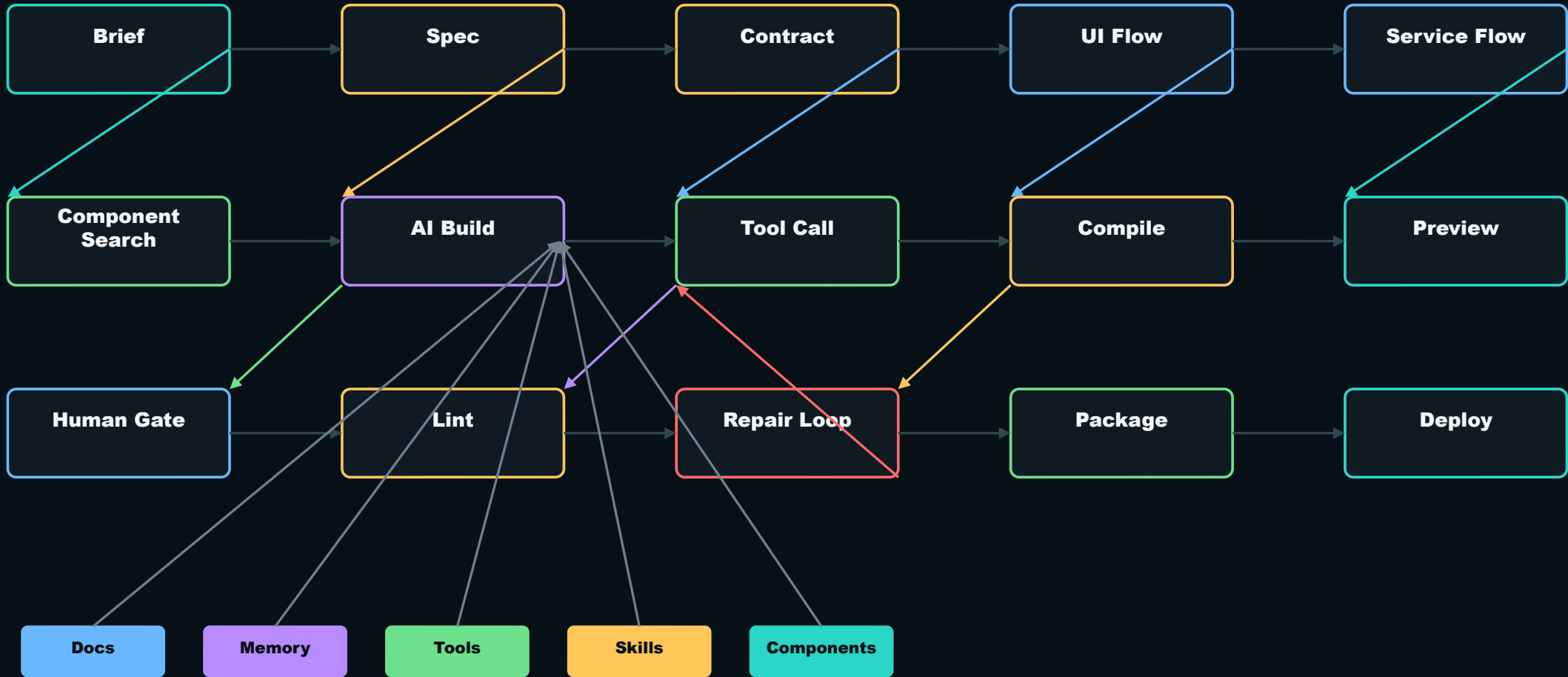
AI becomes one node type in a controlled harness.



Explicit execution order | typed dataflow | checkpoints | retries | human gates

The DAG is the program

A runnable graph captures execution, dataflow, contracts, branches, and repair.



Real VLC Flow mode

Complex DAG workflows are already visible and runnable inside VLC.

The screenshot displays the VLCode Lite v1.33.1 interface, which is a Visual Logic IDE. The main workspace shows a complex DAG (Directed Acyclic Graph) workflow titled "SmartCampusFactoryShowcaseApp.vx" under the project "V21OpsSchemaPack". The workflow is composed of several nodes connected by arrows, representing a sequence of tasks. The nodes include:

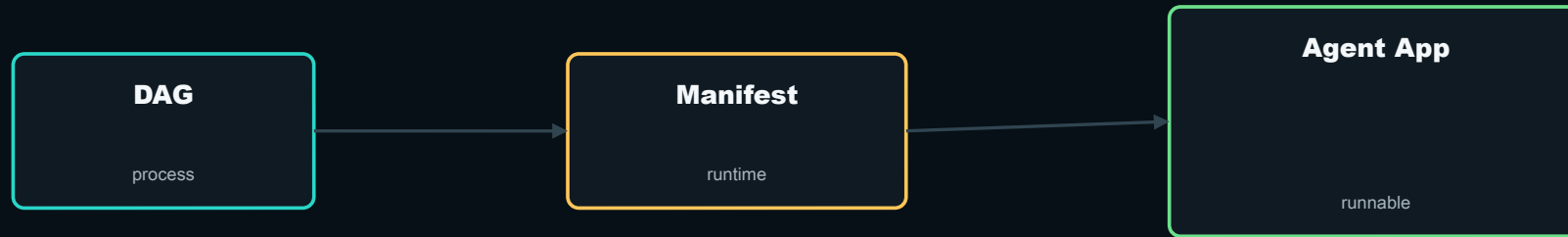
- 1. Crawl the source web...** (Tool: WebsiteCapture)
- 2. Collect screenshots...** (Set: Idle)
- 3. Check whether screen...** (Branch: Idle)
- 4. Create the clone blu...** (Tool: VisionAnalyze)
- 5. Select the downstream...** (Assignment: Sync)
- 6. Run the selected dow...** (Subflow Call: Sync)
- 7b. Continue without vi...** (Assignment: Sync)

The interface also features a left sidebar with a file explorer showing the project structure, including folders like "Apps/" and "Sections/". A top bar contains tabs for "Code", "Flow", "Visual", "Logic", "Data", and "Map". A right sidebar shows the "Object Tree" and "Widgets" panels. The bottom status bar indicates the current state of the application, including "Ready" and "Compile Diagnostics".

A real VisualLogic graph: specialized nodes, visible state, logs, validation, and repair loops.

Agent Apps are DAG plus manifest

A workflow becomes a runnable, shareable application.



Inputs / outputs

Typed entry points

UI panels

Interaction surface

Permissions

Runtime boundaries

Tools / skills

Callable capabilities

Human gates

Review and approval

Packaging

Local or remote run

VL makes the system composable

Structured layers plus components make parallel AI development practical.

Structured layers

Contracts, services, pages, sections, and themes are decoupled. Once the contract is clear, teams and agents can build in parallel.

Pure components

Everything can be composed: UI, flows, tools, skills, services, and theme resources. Components expose methods, properties, and events.

Simple for agents

The language is easy for people and AI to learn. Agents can call components directly instead of guessing hidden behavior.

More reusable components -> faster delivery -> more reliable enterprise adoption.

VisualLogic AI

VL + DAG + AI Assistant

Free local IDE

Download VL-Code and build real projects with visual flows, component resources, tools, skills, and AI assistance.

Open resources

Design and upload flows, components, tools, and skills. Platform resources from VisualLogic are free to use.

Enterprise landing

Designed for complete project generation, not isolated code fragments. Project managers and non-developers can participate directly.

github.com/VisualLogic-AI/VL-Code

github.com/VisualLogic-AI/VisualLogic.ai-VL

